

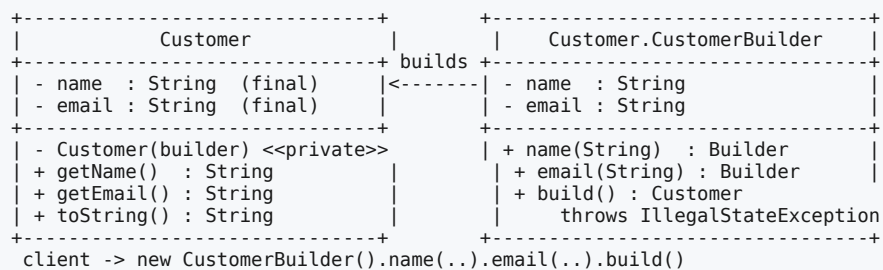
Builder Design Pattern

Intent

Separate the **construction** of a complex object from its representation, so the same building steps can produce an object step by step. In practice, the Builder pattern lets you create an object by naming each field you set — `.name(...).email(...).build()` — instead of passing a long, order-sensitive list of constructor arguments, and it gives the object a single choke point (`build()`) where invariants can be checked before the object is allowed to exist.

This implementation is the classic **static nested builder** form: the product (`Customer`) has a private constructor, and a nested `CustomerBuilder` is the only thing allowed to create it. `build()` also **validates** that the required fields were actually supplied — without that check this would just be a setter chain wearing a Builder's name; the validation is what gives step-by-step construction a real reason to exist here.

UML class diagram



The players

builder/Customer	the product – immutable, private constructor
builder/Customer.CustomerBuilder	the concrete builder – static nested class
BuilderDesignPattern	the client / driver, contains main()

- **Customer** — the finished, immutable object. Every field is `final`; the only constructor is `private`, so the sole path to an instance is `CustomerBuilder.build()`.
- **Customer.CustomerBuilder** — the builder. Holds mutable working copies of the two fields, exposes one fluent setter per field, and validates both are present before handing them to the product's private constructor.
- **BuilderDesignPattern** — the client / driver. Contains `main()`, plays the role of the (implicit) Director: it decides which fields to set and in what order, then calls `build()`.

Code walkthrough — every line explained

Customer.java

```
package com.design.patterns.builder;

public class Customer {

    private final String name;
    private final String email;

    private Customer(CustomerBuilder customerBuilder) {
        this.name = customerBuilder.name;
        this.email = customerBuilder.email;
    }

    public String getName() {
        return name;
    }

    public String getEmail() {
        return email;
    }
}
```

```

public static class CustomerBuilder {
    private String name;
    private String email;

    public CustomerBuilder name(String name) {
        this.name = name;
        return this;
    }

    public CustomerBuilder email(String email) {
        this.email = email;
        return this;
    }

    public Customer build() {
        if (name == null || name.trim().isEmpty()) {
            throw new IllegalStateException("name is required");
        }
        if (email == null || email.trim().isEmpty()) {
            throw new IllegalStateException("email is required");
        }
        return new Customer(this);
    }
}

@Override
public String toString() {
    return "Customer{name=" + name + ", email=" + email + "}";
}
}

```

Line by line:

- `package com.design.patterns.builder;` — places the product and its builder in their own `builder` sub-package, separate from the top-level `com.design.patterns` package that holds the client. This mirrors how `RealUseCase_Builder_Design_Pattern` organizes `builder/DataRetrievalRequest`.
- `public class Customer {` — the **Product**. `public` so the client package can reference it.
- `private final String name; / private final String email;` — the two fields are `private` (no outside code can touch them directly) and `final` (once assigned in the constructor, they can never change). This is what makes a built `Customer` immutable.
- `private Customer(CustomerBuilder customerBuilder) {` — the **only** constructor, and it is `private`. No class outside `Customer` — not even in the same package — can call `new Customer(...)`. The single parameter is the builder itself, which already holds the values to copy in.
- `this.name = customerBuilder.name; / this.email = customerBuilder.email;` — reads the builder's package-private fields directly. This is legal because `CustomerBuilder` is a **nested** class of `Customer`; nested and enclosing classes share access to each other's private members in Java.
- `public String getName() { return name; } / public String getEmail() { return email; }` — read-only accessors. There are deliberately **no setters** — once `build()` returns, nothing can mutate the object.
- `public static class CustomerBuilder {` — the **Concrete Builder**, nested inside `Customer`. `static` is essential: a non-static inner class carries an implicit reference to an *existing* enclosing instance, but there is no `Customer` yet — building one is the whole point. `static` removes that dependency.
- `private String name; / private String email;` — mutable working copies, **not** `final`, because they are filled in incrementally by the fluent setters rather than in one constructor call.
- `public CustomerBuilder name(String name) { this.name = name; return this; }` — stores the value, then returns `this`. Returning the builder itself is the single mechanism that enables method chaining (`.name(...).email(...)`); without it these would return `void` and each call would need its own statement.
- `public CustomerBuilder email(String email) { this.email = email; return this; }` — identical shape for the second field.
- `public Customer build() {` — the terminal step of the chain. Its return type is `Customer`, not `CustomerBuilder`, so the chain cannot continue after this call.
- `if (name == null || name.trim().isEmpty()) { throw new IllegalStateException("name is required"); }` — **validation before construction**. A half-configured `Customer` (missing name) is refused here, before the object is ever allocated. This is the genuine reason this class needs a builder rather than a bare constructor: the check happens once, in one place, regardless of which fields were set in which order.
- `if (email == null || email.trim().isEmpty()) { throw new IllegalStateException("email is required"); }` — the same guarantee for `email`.
- `return new Customer(this);` — only reached once both fields have passed validation; calls the private constructor, which is legal here because `build()` is a member of the nested `CustomerBuilder`.
- `@Override public String toString() { return "Customer{name=" + name + ", email=" + email + "}"; }` — a real, field-formatted `toString()` (not the default `Object.toString()` identity-hash form), so printing a `Customer` shows its actual data — used directly by the demo's `System.out.println(customer)`.

BuilderDesignPattern.java

```

package com.design.patterns;

import org.springframework.boot.SpringApplication;

```

```
import org.springframework.boot.autoconfigure.SpringBootApplication;

import com.design.patterns.builder.Customer;

@SpringBootApplication
public class BuilderDesignPattern {

    public static void main(String[] args) {
        SpringApplication.run(BuilderDesignPattern.class, args);

        Customer customer = new Customer.CustomerBuilder().name("Avinash
Patel").email("infinityDoesExist@gmail.com")
            .build();
        System.out.println(customer);
    }
}
```

Line by line:

- `package com.design.patterns;` — the top-level package for this module; the client sits one level above the `builder` sub-package it consumes.
- `import com.design.patterns.builder.Customer;` — brings `Customer` (and, transitively, its nested `CustomerBuilder`) into scope by simple name.
- `@SpringBootApplication` — marks this as a Spring Boot entry point so `SpringApplication.run(...)` can bootstrap a context; unrelated to the Builder pattern itself, purely infrastructure for the demo.
- `public class BuilderDesignPattern {` — named after the pattern, per this repo's convention (`FacadeDesignPattern`, `AbstractFactoryDesignPattern`, etc.), and matching its own filename as Java requires for a public top-level class.
- `public static void main(String[] args) {` — the JVM entry point.
- `SpringApplication.run(BuilderDesignPattern.class, args);` — boots the Spring context; not required by the pattern, only by this module's choice of Spring Boot as a runner.
- `Customer customer = new Customer.CustomerBuilder().name("Avinash Patel").email("infinityDoesExist@gmail.com").build();` — creates an empty builder, chains `.name(...)` then `.email(...)` (each returning the same builder instance), then calls `.build()`, which validates both fields are present and constructs the immutable `Customer`.
- `System.out.println(customer);` — implicitly calls `customer.toString()`, printing the field-formatted representation.

Why these design decisions

Why a builder at all — what problem does it solve?

Two classic problems with plain constructors:

1. **The telescoping constructor.** With many optional fields you end up with a pile of overloads — `Customer(name)`, `Customer(name, email)`, `Customer(name, email, phone)`, ... — that is painful to write and read.
2. **Order-sensitive, unreadable calls.** `new Customer("Avinash", "a@b.com")` — which argument is which? Easy to swap two `Strings` by accident and the compiler won't notice. The builder replaces this with **self-documenting, named** calls: `.name("Avinash").email("a@b.com")`. Order no longer matters and each value is labeled.

Why does `build()` validate instead of just calling the constructor?

Without validation, `CustomerBuilder` would be nothing more than a setter chain that happens to end in `.build()` — the "Builder" name would not be earning its keep, because a two-argument constructor would do exactly the same job with less code. The validation gives `build()` real responsibility: it is the single point where the object's invariants (a `Customer` must have a name and an email) are enforced, regardless of what order the setters were called in or whether one was skipped entirely. That is a genuine reason step-by-step construction — accumulate, then validate, then produce — earns its place over a plain constructor.

Why the private constructor?

To **guarantee** the builder is the only path in. If the constructor were public, callers could bypass the builder — and its validation — and construct a half-configured object directly. Sealing it makes the builder mandatory.

Why `return this;` in each setter?

That single line is the entire mechanism behind the fluent chain. Without it the setters would return `void` and you'd be forced to write:

```
CustomerBuilder b = new CustomerBuilder();
b.name("Avinash");
```

```
b.email("a@b.com");
Customer c = b.build();
```

Returning `this` collapses that into one readable expression.

Why is the builder `static`?

Because the builder must be usable **without** an existing `Customer`. A non-static inner class is tied to an outer instance (`outer.new Inner()`), but here there is no `Customer` yet — that's the whole point of building one. `static` breaks that dependency and also avoids leaking a reference to a non-existent enclosing object.

Why immutability (no setters on `Customer`)?

Once `build()` returns, the `Customer` cannot be modified — there are no setters and the fields are private and final. Immutable objects are inherently **thread-safe** (no one can change them under you), safe to share and cache, and safe to use as map keys. The mutable, "still being configured" state lives only in the throwaway builder; the finished product is frozen.

Manual builder vs. Lombok `@Builder`

This module writes the builder **by hand** to make the mechanism — and the validation — visible. Lombok's `@Builder` annotation would generate the nested builder, the fluent setters, and `build()` automatically, but it would not add the required-field validation for free; that logic still has to be written by hand (typically with `@Builder` plus a custom `build()` override, or `@NonNull` fields). The hand-written version here is the teaching version — it shows exactly what a generated builder produces, plus the invariant check a production builder usually needs.

Execution flow (as run from `main`)

```
BuilderDesignPattern.main
├── SpringApplication.run(...)           boots Spring (not required by the pattern)
├── new Customer.CustomerBuilder()       create an empty builder
│   ├── .name("Avinash Patel")          stores name, returns the builder
│   ├── .email("infinityDoesExist@gmail.com") stores email, returns the builder
│   └── .build()
│       ├── name is non-blank → passes validation
│       ├── email is non-blank → passes validation
│       └── new Customer(builder) → copies fields in
│           └── returns an immutable Customer
│               └── toString() → "Customer{name=Avinash Patel,
email=infinityDoesExist@gmail.com}"
```

Expected output

```
Customer{name=Avinash Patel, email=infinityDoesExist@gmail.com}
```

Captured from a real run (`java -jar target/Builder_Design_Pattern-0.0.1-SNAPSHOT.jar`); the Spring Boot startup/shutdown log lines are omitted above for readability.

How to run

```
# From this module's directory
JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 mvn -o clean package -DskipTests

# Run the packaged Spring Boot jar (it bundles every dependency, unlike
# plain `target/classes`, which is missing Spring on its classpath)
java -jar target/Builder_Design_Pattern-0.0.1-SNAPSHOT.jar

# The app boots an embedded web server (spring-boot-starter-web) and stays
# up after printing the line above – stop it with Ctrl-C, or pass
# --server.port=0 to bind an ephemeral port instead of 8080.
```

Drop `-o` (offline) on a first build if the parent reactor's dependencies are not yet in your local Maven repository.

Relationship to the other Creational patterns

- **Factory / Abstract Factory** decide **which class** to instantiate.
- **Builder** is about **how to assemble one object** that has many parts/fields — it doesn't choose a type, it constructs a single complex instance step by step, and can enforce invariants (like required fields) before the object is allowed to exist.
- **Prototype** avoids construction entirely by copying an existing instance.

Use a Builder when an object has several fields (especially optional ones, or ones that need validating together) and you want construction to be readable, order-independent, and to yield an immutable result.

Note: `SpringApplication.run(...)` just boots a Spring context and is unrelated to the pattern; the `Customer` here is built manually, not managed by Spring.